

Tune3

Otimização de hiperparâmetros consciente de curvatura para detecção de malware

Um guia autocontido: do problema à implementação

Projeto Tune3 — coordenação: Prof. Dionatan R. Schmidt
UNIPAMPA, Campus Alegrete — AI Horizon Labs

Versão de 16 de setembro de 2026 — alinhada ao commit pós-auditoria do repositório
`dionatanschmidt/tune3`

Resumo

Este documento explica, sem pressupor leitura prévia, o que o Tune3 faz, por que faz e como cada peça está implementada. Ele foi escrito para uma pessoa que queira *entender* o projeto — um aluno novo, um colaborador de outra área ou um revisor — e não para vender uma tese: onde a evidência é forte, dizemos; onde ela é fraca ou inexistente, também dizemos. Toda afirmação sobre o código refere-se ao estado do repositório após a auditoria de setembro de 2026; toda afirmação sobre a literatura refere-se ao que os artigos de fato estabelecem, conforme a leitura corrigida do próprio projeto.

Sumário

1	Como ler este documento	3
2	O problema: a média não protege ninguém	3
2.1	Hiperparâmetros versus parâmetros	3
3	Risco operacional: da média ao CVaR	3
3.1	O risco operacional completo	4
3.2	Por que não estimar tudo por Monte Carlo	4
4	A aposta geométrica: curvatura como proxy de risco	4
4.1	Hessiana, autovalores e a paisagem de perda	4
4.2	A grandeza escolhida: $\text{Tr}(H^2)$	4
5	Estimando $\text{Tr}(H^2)$: o estimador de Hutchinson	5
5.1	A identidade básica	5
5.2	Como obter Hv sem formar H	5
6	A segunda observação: razão sinal-ruído do gradiente (GSNR)	6
7	Nível MACRO: busca bayesiana bi-objetivo	7
7.1	O problema bi-objetivo	7
7.2	Otimização bayesiana com processos gaussianos	7
7.3	Dois regras de seleção na fronteira	7

8	Nível MICRO: o controlador DDKF	7
8.1	O que o DDKF controla	7
8.2	O que o DDKF observa	8
8.3	Como o DDKF corrige	8
8.4	Identificação exige excitação — e o gating por regime	8
9	Guardas de segurança	9
9.1	Aborto robusto por Cantelli	9
9.2	Detector de <i>edge of stability</i>	9
10	O algoritmo completo	10
11	Comparadores e desenho experimental	11
11.1	Dados e cenários	11
11.2	Os oito métodos	11
11.3	Protocolo estatístico (pré-registrado)	11
12	Estado da evidência e a tensão científica	12
12.1	O que foi observado (com o pipeline anterior)	12
12.2	As três rachaduras na explicação geométrica	12
12.3	H_G contra H_M	12
12.4	Linha 1: E0, E1, E2 e a árvore de decisão	12
13	Literatura: o que cada artigo sustenta — e o que não sustenta	13
14	Glossário de símbolos	14
A	Mapa teoria $\leftrightarrow$ código	14

1 Como ler este documento

O texto avança em camadas. As Seções 2 a 4 constroem a motivação (por que *risco de cauda*, por que *curvatura*). As Seções 5 a 9 explicam cada componente matemático com a mesma notação que aparece no código. A Seção 10 monta o algoritmo completo. A Seção 12 apresenta o estado da evidência e a tensão científica que a próxima rodada de experimentos precisa resolver. A Seção 13 lista o que cada artigo do corpus sustenta — e o que não sustenta. O Apêndice A mapeia cada símbolo a um arquivo e uma classe.

Convenção de honestidade

Usamos três rótulos ao longo do texto. **RESULTADO**: algo demonstrado ou verificado computacionalmente. **HIPÓTESE**: algo que o projeto *aposta* e ainda está testando. **DECISÃO DE PROJETO**: uma escolha de engenharia que poderia ter sido outra. Confundir os três é o erro mais comum em textos sobre este tipo de método.

2 O problema: a média não protege ninguém

Um detector de malware é, do ponto de vista estatístico, um classificador f_θ com parâmetros $\theta \in \mathbb{R}^W$ treinado para minimizar o *risco empírico*

$$\min_{\theta} \mathbb{E}_{(x,y) \sim P} [L(f_\theta(x), y)],$$

onde L é uma perda (aqui, entropia cruzada ponderada por classe). Essa formulação otimiza o *caso médio*. Um centro de operações de segurança (SOC), porém, não é avaliado pela média: ele é avaliado pelo pior caso — o dia em que uma família nova de malware passa despercebida ou em que um lote de aplicativos legítimos é bloqueado. Duas configurações com a mesma perda média podem ter comportamentos operacionais opostos: uma com perdas concentradas perto da média, outra com uma cauda longa de erros graves e raros.

O Tune3 parte dessa observação e troca o alvo: em vez de perguntar “qual configuração de hiperparâmetros minimiza a perda média?”, pergunta “qual configuração minimiza a *cauda* da distribuição de perdas?”.

2.1 Hiperparâmetros versus parâmetros

Vamos manter a distinção clara desde já. $\theta \in \mathbb{R}^W$ são os *pesos* da rede, ajustados por descida de gradiente. $h \in \mathcal{H} \subset \mathbb{R}^d$ são os *hiperparâmetros* (taxa de aprendizado, decaimento de peso, largura, profundidade, dropout), com $d \ll W$; no Tune3, $d = 5$ e W está na casa das dezenas de milhares. A otimização de hiperparâmetros (HPO) escolhe h ; cada avaliação de h custa um treinamento completo. Por isso HPO é caro e por isso a escolha de *o que* medir em cada treinamento importa tanto.

3 Risco operacional: da média ao CVaR

Definição 3.1 (VaR e CVaR). Seja ℓ uma variável aleatória de perda e $\gamma \in (0, 1)$. O *Value-at-Risk* é o γ -quantil, $\text{VaR}_\gamma(\ell) = \inf\{t : \Pr[\ell \leq t] \geq \gamma\}$, e o *Conditional Value-at-Risk* é a perda esperada na cauda além dele:

$$\text{CVaR}_\gamma(\ell) = \mathbb{E}[\ell \mid \ell \geq \text{VaR}_\gamma(\ell)].$$

Com $\gamma = 0,95$, o CVaR é a média dos 5% piores casos. Ele é *coerente* no sentido de Artzner et al. (subaditivo, monótono, homogêneo), o que o VaR não é, e admite a representação variacional de Rockafellar e Uryasev [1]:

$$\text{CVaR}_\gamma(\ell) = \min_{\tau \in \mathbb{R}} \left\{ \tau + \frac{1}{1-\gamma} \mathbb{E}[(\ell - \tau)_+] \right\}, \quad \text{com ótimo em } \tau^* = \text{VaR}_\gamma(\ell). \quad (1)$$

Exemplo 3.1. Perdas por amostra $\{0,1; 0,1; 0,2; 0,2; 0,3; 0,3; 0,4; 0,5; 2,0; 9,0\}$ (média 1,31). Com $\gamma = 0,8$, o VaR é 2,0 e o CVaR é a média de $\{2,0; 9,0\} = 5,5$. Um segundo conjunto com a mesma média mas sem a cauda ($\{1,31\} \times 10$) tem CVaR = 1,31. A média não distingue os dois; o CVaR distingue por um fator de quatro.

No código

`tune3/core/objectives.py` implementa as duas formas — `cvar_quantile` (definição direta) e `cvar_rockafellar` (Eq. 1) — e um teste verifica que concordam (`test_cvar_formulas_concordam`). O CVaR é calculado sobre as *perdas por amostra* do conjunto de validação (durante a busca) ou de teste (na avaliação final), com $\gamma = 0,95$.

3.1 O risco operacional completo

O projeto formaliza o risco de uma configuração h como

$$R(h) = \mathbb{E}_P[L(\theta_h)] + \alpha \text{Var}_P(L(\theta_h)) + \beta \text{CVaR}_\gamma(L(\theta_h)), \quad (2)$$

em que θ_h é o resultado do treinamento sob h e $\alpha, \beta \geq 0$ são pesos de segurança. Na prática, o Tune3 usa o CVaR como objetivo primário e trata a variância indiretamente, pela via geométrica da próxima seção.

3.2 Por que não estimar tudo por Monte Carlo

Estimar $\text{Var}_P(L(\theta_h))$ e CVaR_γ com precisão exigiria repetir o treinamento sob h muitas vezes (variando dados e sementes) para cada h candidato. Se a busca avalia 28 configurações e cada estimativa precisa de 30 treinamentos, são 840 treinamentos por experimento por semente. É aqui que entra a aposta central do projeto.

4 A aposta geométrica: curvatura como proxy de risco

4.1 Hessiana, autovalores e a paisagem de perda

A Hessiana da perda em relação aos pesos, $H_\theta = \nabla_\theta^2 L(\theta) \in \mathbb{R}^{W \times W}$, descreve a curvatura local da *paisagem de perda*. Seus autovalores $\lambda_1 \geq \dots \geq \lambda_W$ medem quão rápido a perda muda ao perturbar θ nas direções principais. Um mínimo “plano” tem autovalores pequenos: perturbar os pesos quase não altera a perda. Um mínimo “agudo” tem autovalores grandes.

Dois fatos empíricos sobre H_θ em redes profundas orientam o projeto (Seção 13 dá as fontes e o escopo):

- o espectro é *degenerado*: a maioria dos autovalores está concentrada perto de zero, com poucos *outliers* grandes (Sagun, Bottou e LeCun);
- o treinamento por descida de gradiente tende a aumentar a curvatura até um limite (“*progressive sharpening*” até a “*edge of stability*”, Cohen et al.).

4.2 A grandeza escolhida: $\text{Tr}(H^2)$

O Tune3 mede a curvatura por

$$\text{Tr}(H_\theta^2) = \sum_{i=1}^W \lambda_i^2 = \|H_\theta\|_F^2,$$

o quadrado da norma de Frobenius da Hessiana. Três razões para essa escolha em vez de $\text{Tr}(H) = \sum \lambda_i$ ou de $\lambda_{\max}$:

- (i) $\text{Tr}(H)$ pode ser pequeno com autovalores grandes de sinais opostos se cancelando; $\text{Tr}(H^2)$ não cancela nada.
- (ii) $\text{Tr}(H^2)$ é dominado pelos outliers (λ_i^2 amplifica os grandes), o que, dado o espectro degenerado, o torna um “sensor de explosão de curvatura” barato.
- (iii) É estimável *sem formar a Hessiana*, por produtos Hessiana-vetor (Seção 5).

Status epistêmico — leia com atenção

A afirmação “vales geometricamente planos limitam a variância da perda sob ruído estocástico e, portanto, suprimir $\text{Tr}(H^2)$ reduz $\text{Var}(L)$ e o CVaR” é a **HIPÓTESE** central do projeto (chamada de H_G na Seção 12). Ela *não* está estabelecida pela evidência interna do projeto até agora, e parte dessa evidência aponta contra a versão forte. Nenhum documento do projeto pode tratá-la como resultado.

Há ainda uma ressalva teórica conhecida (Dinh et al., 2017): em redes ReLU, a “planura” não é invariante a reparametrizações que reescalas camadas adjacentes; dois pontos com a mesma função podem ter Hessianas muito diferentes. Qualquer afirmação sobre “mínimo mais plano” precisa, portanto, fixar a parametrização — o Tune3 compara sempre a mesma arquitetura, com a mesma parametrização, e mede $\text{Tr}(H^2)$ como *objetivo relativo* entre configurações, não como grandeza absoluta.

5 Estimando $\text{Tr}(H^2)$: o estimador de Hutchinson

5.1 A identidade básica

Proposição 5.1 (Hutchinson, 1990). Seja $A \in \mathbb{R}^{W \times W}$ e v um vetor aleatório com componentes independentes, $\mathbb{E}[v] = 0$ e $\mathbb{E}[vv^\top] = I$ (por exemplo, Rademacher: $v_i = \pm 1$ com probabilidade $1/2$). Então $\mathbb{E}[v^\top A v] = \text{Tr}(A)$ e, em particular, $\mathbb{E}[\|A v\|^2] = \text{Tr}(A^\top A)$.

Demonstração. $\mathbb{E}[v^\top A v] = \mathbb{E}[\sum_{ij} v_i A_{ij} v_j] = \sum_{ij} A_{ij} \mathbb{E}[v_i v_j] = \sum_i A_{ii} = \text{Tr}(A)$. Para a segunda parte, $\|A v\|^2 = v^\top A^\top A v$ e aplica-se o primeiro resultado a $A^\top A$. $\square$

Com $A = H_\theta$ simétrica, $\text{Tr}(H^\top H) = \text{Tr}(H^2)$. Logo, com M sondas independentes $v_1, \dots, v_M$,

$$\widehat{\text{Tr}}(H^2) = \frac{1}{M} \sum_{i=1}^M \|H v_i\|^2 \quad (3)$$

é um estimador não-viesado, com desvio-padrão proporcional a $1/\sqrt{M}$.

5.2 Como obter Hv sem formar H

O produto Hessiana-vetor (HVP) é a derivada direcional do gradiente:

$$Hv = \nabla_\theta (\nabla_\theta L(\theta)^\top v).$$

Em PyTorch: calcula-se $g = \nabla_\theta L$ *uma vez*, com `create_graph=True` (para poder diferenciar g de novo), forma-se o escalar $g^\top v$ e diferencia-se em relação a θ . Cada HVP custa cerca de um *backward*; o estimador completo custa 1 forward + 1 backward de primeira ordem + M HVPs — $O(M+1)$ backwards, não $O(2M)$.

No código — tune3/curvature/hutchinson.py

```

grads = torch.autograd.grad(loss, params, create_graph=True, retain_graph=True) #
    1x, fora do laço
for i in range(M):
    v = [rademacher_like(p) for p in params]
    gv = sum(torch.sum(g * vi) for g, vi in zip(grads, v)) # g^T v
    Hv = torch.autograd.grad(gv, params, retain_graph=(i < M - 1)) # H v
    acc += float(sum(torch.sum(h * h) for h in Hv)) # ||Hv||^2
tr = acc / M

```

Verificação independente (setembro de 2026): num MLP com 194 parâmetros, o Hv desse código coincide com Hv calculado a partir da Hessiana exata (`torch.autograd.functional.hessian`) até $1,5 \times 10^{-7}$, e a média de 20 000 sondas converge para $\|H\|_F^2$ exato ($z = -0,77$). O padrão $M = 5$ é usado dentro do treinamento.

Por que o estimador roda com `model.eval()`

Com dropout ativo, cada forward avalia uma *sub-rede aleatória*; a Hessiana medida é a dessa sub-rede, e a média sobre sondas mistura sub-redes diferentes. Na auditoria, isso inflou a estimativa em 28–32% num MLP de teste. O trial do Tune3 desliga o dropout durante a estimativa e o religa em seguida.

6 A segunda observação: razão sinal-ruído do gradiente (GSNR)

Definição 6.1 (GSNR, Liu et al. 2020). Para o gradiente estocástico $\hat{g} = \nabla_{\theta} L_{\text{minibatch}}$ com média $\mathbb{E}[\hat{g}] = \bar{g}$ e covariância Σ , a razão sinal-ruído agregada é

$$\text{GSNR} = \frac{\|\bar{g}\|^2}{\text{Tr}(\Sigma)}.$$

GSNR alto significa que minibatches diferentes “concordam” sobre a direção de descida — pode-se dar passos maiores. GSNR baixo significa gradiente dominado por ruído de amostragem — passos menores.

No código — tune3/curvature/gsnr.py

O estimador é o *direto*: guarda os gradientes achatados dos $K = 4$ primeiros minibatches de cada época, calcula $\bar{g}$ e a variância amostral componente a componente (`ddof=1`) e devolve $\|\bar{g}\|^2 / \sum_i \widehat{\text{Var}}[\hat{g}_i]$. O DDKF (Seção 8) recebe $\log \text{GSNR}$ (com piso 10^{-8}).

Observação 6.1 (Documentação antiga vs. código). O manuscrito teórico descrevia um estimador por *duplo EMA* em $O(1)$ por passo. O código *nunca* usou EMA; usa o estimador direto acima. A correção “C3” dos documentos antigos (remover uma divisão redundante por B) não se aplica a este código. **DECISÃO DE PROJETO**: manter o estimador direto e corrigir o manuscrito.

Uma consequência verificável. Para Σ fixa, a covariância do gradiente de um minibatch de tamanho B é Σ/B , logo $\text{GSNR} \propto B$ e $\partial \text{GSNR} / \partial B = \text{GSNR} / B$. A validação do repositório mede $\text{GSNR}(B=1024) / \text{GSNR}(B=128) \approx 14,5$ (teoria: 8), coerente com a proporcionalidade dentro do ruído de estimação. Esse é o argumento para tratar B como hiperparâmetro *estático* no Tune3: mudar B durante o treino move o GSNR de forma previsível e ao mesmo tempo perturba a alocação de memória — não vale o custo.

7 Nível MACRO: busca bayesiana bi-objetivo

7.1 O problema bi-objetivo

Para cada configuração h , um treinamento devolve dois números: $y_1(h) = \text{CVaR}_{0,95}$ das perdas de validação (na melhor época) e $y_2(h) = \widehat{\text{Tr}}(H^2)$ ao final. O Tune3 minimiza os dois simultaneamente. Como em geral não existe h que minimize ambos, o resultado é uma *fronteira de Pareto*: o conjunto das configurações não dominadas (nenhuma outra é melhor nos dois objetivos ao mesmo tempo).

7.2 Otimização bayesiana com processos gaussianos

Cada avaliação de h é cara (um treinamento), então o Tune3 constrói um modelo substituto: um processo gaussiano (GP) independente por objetivo, ajustado aos pares (h, y_j) já observados, com entradas normalizadas ao cubo unitário e saídas padronizadas. O GP fornece, para qualquer h não avaliado, uma média e uma incerteza. A *função de aquisição* decide qual h avaliar em seguida, equilibrando explorar (alta incerteza) e refinar (boa média).

A aquisição usada é o *Expected Hypervolume Improvement* na forma logarítmica e robusta a ruído (`qLogNoisyExpectedHypervolumeImprovement`, BoTorch): escolhe-se o h que mais aumenta, em expectativa, o hipervolume dominado pela fronteira de Pareto atual em relação a um ponto de referência adaptativo. Sequência: 8 pontos iniciais de Sobol, depois 20 iterações de aquisição — 28 treinamentos por busca.

7.3 Duas regras de seleção na fronteira

Ao final, a fronteira contém tipicamente de dois a seis pontos. O projeto reporta *duas* escolhas, e a distinção é central nos resultados:

- **knee point**: o ponto que minimiza a soma dos objetivos normalizados — o “equilíbrio” entre CVaR e curvatura (a configuração mais plana entre as razoáveis);
- **best-CVaR**: o ponto da fronteira com menor CVaR — alinhado à métrica de implantação.

Observação 7.1 (O que o MACRO *não* tem). O manuscrito menciona “cEHVI” com uma restrição operacional L^* (bloquear regiões em que a perda média prevista viola um SLA). O código usa EHVI *sem* restrição, e um trial que aborta devolve a penalidade $(10^3, 10^3)$ em vez de (∞, ∞) . O GP assume ruído homoscedástico. São **DECISÕES DE PROJETO** documentadas na auditoria; a restrição L^* e o ruído heteroscedástico são melhorias pendentes.

8 Nível MICRO: o controlador DDKF

8.1 O que o DDKF controla

Dentro de cada treinamento, a taxa de aprendizado η_t é ajustada época a época. O estado do controlador é $x_t = \log \eta_t$ e sua dinâmica nominal é um processo AR(1) em torno de um valor nominal x_{nom} :

$$x_{t+1} = x_{\text{nom}} + \rho(x_t - x_{\text{nom}}) + w_t, \quad w_t \sim \mathcal{N}(0, \sigma_\eta^2), \quad \rho = 0,95. \quad (4)$$

Sem observações, x_t relaxa para x_{nom} com meia-vida $\log 0,5 / \log 0,95 \approx 13,5$ épocas. A variância estacionária é $S_\infty = \sigma_\eta^2 / (1 - \rho^2)$.

Observação 8.1 (x_{nom} é constante). No código, $x_{\text{nom}} = \log \eta_0$ (a taxa sugerida pelo MACRO) e *não muda* durante o treino: não há scheduler base (Cosine, step) por baixo. O manuscrito descrevia o DDKF como um “meta-scheduler” que filtra desvios em torno de um scheduler nominal $\eta_{\text{sched}}(t)$; isso *não* está implementado. **DECISÃO DE PROJETO** a registrar.

8.2 O que o DDKF observa

A cada época, o controlador recebe o vetor de observação

$$z_t = (L_t^{\text{treino}}, L_t^{\text{val}}, \log \text{GSNR}_t)^\top \in \mathbb{R}^3. \quad (5)$$

A curvatura *não* entra no DDKF

O manuscrito teórico especifica $z_t = (L, \log_{10}(1 + \text{Tr}(H^2)), \log \text{GSNR})$. O manual de implementação e o código usam a Eq. (5): perda de treino, perda de validação e GSNR. A curvatura atua no nível MACRO (como objetivo) e no detector de EoS (Seção 9), mas o filtro de Kalman não a vê. Todos os resultados obtidos até hoje foram produzidos com a Eq. (5). Este documento descreve o código; a decisão de manter ou alterar o vetor de observação é do coordenador (item D1 da auditoria).

8.3 Como o DDKF corrige

Diferentemente de um filtro de Kalman clássico, que rastreia um estado externo desconhecido, aqui o estado x_t é a variável de controle. O filtro identifica empiricamente, numa janela deslizando de até 100 épocas, a covariância entre o que ele fez (x) e o que observou (z):

$$P_{xz} = \widehat{\text{Cov}}(x, z) \in \mathbb{R}^{1 \times 3}, \quad P_{zz} = \widehat{\text{Cov}}(z, z) \in \mathbb{R}^{3 \times 3} \text{ (com encolhimento isotrópico de 10\% e piso).}$$

O ganho é $K = P_{xz}P_{zz}^{-1} \in \mathbb{R}^{1 \times 3}$ e a correção é o escalar

$$u_t = K(z_t - \bar{z}), \quad x_{t+1} = [x_{\text{nom}} + \rho(x_t - x_{\text{nom}})] - u_t,$$

com $|u_t|$ limitado a $3\sqrt{S_\infty}$. A interpretação: se historicamente “subir o lr” veio acompanhado de “perda maior” ($\text{Cov} > 0$) e a perda atual está *acima* da média recente, então $u_t > 0$ e o lr desce; se a perda está abaixo da média, o lr sobe um pouco. O DDKF é, portanto, um **regulador de desvios**: ele não procura o lr ótimo, ele reage a observações que se afastam do padrão recente.

8.4 Identificação exige excitação — e o gating por regime

Para estimar $\text{Cov}(x, z)$ é preciso que x tenha variado. Por isso, durante o aquecimento (as primeiras 30 épocas), o controlador injeta ruído de exploração ($\sigma = 0,03$ em $\log \eta$). Depois disso, com a configuração padrão (`exploration_decay=0`), a excitação cessa.

A qualidade da identificação é medida pela *razão de informação*

$$R^2 = \frac{S_u}{S_\infty}, \quad S_u = P_{xz}P_{zz}^{-1}P_{xz}^\top,$$

a fração da variância estacionária do estado explicada pelas observações. O **RegimeGate** classifica cada época: regime A ($R^2 < 0,10$): observação pouco informativa, *não aplica* a correção (predict-only); regime B ($0,10 \leq R^2 < 0,50$): filtra; regime C ($R^2 \geq 0,50$): filtra com alta confiança.

O que a telemetria mostra

Com os padrões atuais, após o aquecimento R^2 tende a zero (a excitação parou, x deixa de variar, $P_{xz} \rightarrow 0$) e o filtro permanece no regime A. Na validação do repositório, o DDKF esteve ativo em 0 de 85 épocas. O próprio código descreve o DDKF como “rede de segurança, não otimizador principal”. Se isso se confirmar em escala (experimento E2, Seção 12.4), o nível MICRO é, na prática, inerte e o relatório precisa dizê-lo. Para tornar esse dado visível, cada trial devolve agora `n_ddkf_active_epochs`, `regime_fractions` e `lr_change_fraction`.

9 Guardas de segurança

9.1 Aborto robusto por Cantelli

Um trial com lr ruim pode divergir e desperdiçar orçamento. Em vez de um limiar mágico, o Tune3 usa a desigualdade unilateral de Cantelli: para qualquer variável com média μ e desvio σ finitos,

$$\Pr[X - \mu \geq k\sigma] \leq \frac{1}{1 + k^2}.$$

Igualando $1/(1 + k^2) = 1 - \gamma$ obtém-se $k_\gamma = \sqrt{\gamma/(1 - \gamma)}$; com $\gamma = 0,95$, $k = \sqrt{19} \approx 4,36$. O limiar de aborto é

$$\Theta_t = \widehat{\text{loc}}_t + k_\gamma \widehat{\text{esc}}_t,$$

com localização robusta (*mediana*) e escala robusta ($\text{MAD} \times 1,4826$, com piso), calculadas sobre a janela das perdas de validação anteriores. A perda atual é testada *antes* de entrar na janela, para que um pico não infle o próprio limiar. A garantia é livre de distribuição: a taxa de alarme falso é no máximo 5% para qualquer distribuição com variância finita (a validação mede 0,9% sob t_3). O uso de mediana/MAD em vez de média/desvio troca a interpretação literal de (μ, σ) por robustez a outliers históricos.

9.2 Detector de *edge of stability*

Cohen et al. mostraram (em descida de gradiente *full-batch*) que a curvatura cresce durante o treino até um limite ligado a $2/\eta$, além do qual a perda oscila. O **EoSDetector** monitora a série de $\widehat{\text{Tr}}(H^2)$ e, se detectar crescimento monotônico sustentado (≥ 4 subidas consecutivas) com razão recente/base $\geq 1,5$, multiplica o lr por 0,7, com um período de espera de 5 estimativas entre disparos.

O bug que anulava o detector

Até a auditoria, o detector recebia o *mesmo* valor de curvatura repetido entre reestimativas (a curvatura só é reestimada a cada `curvature_every= 5` épocas). Como a regra exige subidas *estritas*, ele nunca disparou em nenhum experimento anterior. Agora ele só recebe valores novos, e conta “passos” em unidades de estimativas. Com `curvature_every= 5`, precisa de 30 épocas de histórico para poder disparar; a paciência de parada antecipada é 10. A escolha entre `curvature_every= 5` e 1 é uma decisão de protocolo em aberto.

10 O algoritmo completo

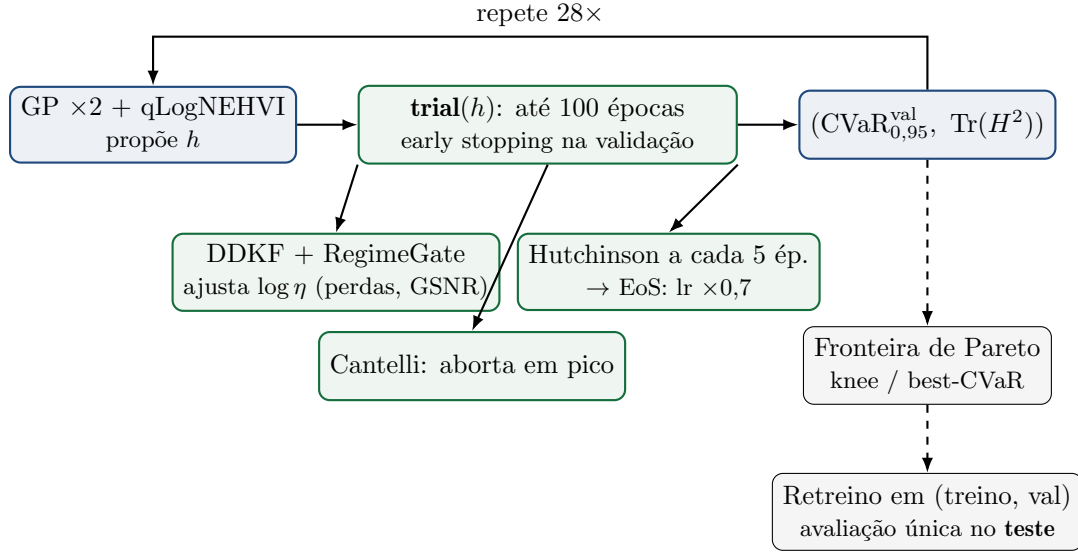


Figura 1: Visão geral do Tune3. À esquerda, o nível MACRO propõe hiperparâmetros e recebe dois objetivos por treinamento. Dentro de cada treinamento (nível MICRO), o DDKF ajusta a taxa de aprendizado a partir de perdas e GSNR; a curvatura alimenta o objetivo e o detector de EoS; o Cantelli aborta divergências. A avaliação final retreina a configuração escolhida usando *apenas* treino e validação e toca no teste uma única vez.

Algoritmo 1: Um trial do Tune3 (`tune3/integration/trial.py`)

Input: $h = (\eta_0, \lambda_{wd}, p_{drop}, \text{largura}, \text{profundidade})$; treino D_{tr} ; validação D_{val}

- 1 inicializar MLP(h), SGD(η_0 , momento 0,9, λ_{wd}), DDKF($x_0 = \log \eta_0$), RegimeGate, Cantelli, EoS, Hutchinson($M = 5$);
- 2 **for** época $t = 0, 1, \dots, 99$ **do**
- 3 treinar uma época em D_{tr} (lotes de 4096); guardar os gradientes dos 4 primeiros lotes;
- 4 $L_t^{tr} \leftarrow$ perda média de treino; **se** não finita: abortar;
- 5 L_t^{val} , perdas por amostra $\ell_t^{val} \leftarrow$ avaliar D_{val} em `eval()`;
- 6 **if** $t \bmod 5 = 0$ **ou** última época **then**
- 7 $c_t \leftarrow \widehat{\text{Tr}}(H^2)$ num lote de treino, em `eval()`; eos $\leftarrow$ EoS.update(c_t);
- 8 $z_t \leftarrow (L_t^{tr}, L_t^{val}, \log \text{GSNR}_t)$; DDKF.update(z_t); $R^2 \leftarrow$ razão de informação; $\phi \leftarrow$ gating(R^2) $\in \{0, 1\}$;
- 9 $\log \eta \leftarrow (1 - \phi) \log \eta_{\text{atual}} + \phi \hat{x}_t$; **se** eos: $\eta \leftarrow 0,7 \eta$;
- 10 **se** Cantelli.should_abort(L_t^{val}): abortar;
- 11 **se** L_t^{val} melhorou ($> 10^{-4}$): guardar (ℓ_t^{val}, c_t) como melhor época; zerar paciência; **senão** paciência++;
- se** paciência ≥ 10 : parar;

Output: $\text{CVaR}_{0,95}(\ell_{\text{melhor}}^{val})$, c_{melhor} , telemetria (regimes, épocas com DDKF ativo, disparos do EoS)

Avaliação final e a separação treino/validação/teste. Para cada método (Tune3 e comparadores), a configuração escolhida é *retreinada* com um trial simples (sem DDKF) em (treino, validação): a melhor época é escolhida pela validação, o estado dessa época é restaurado e só então o modelo é avaliado no teste, uma única vez. O CVaR de teste dessa avaliação é a métrica primária. Isso implica algo que precisa ficar claro: *o DDKF influencia a escolha dos hiperparâmetros, não o modelo final implantado.*

O que estava errado antes

Até setembro de 2026, o retreino final recebia o conjunto de *teste* no lugar da validação: a melhor época era escolhida olhando o teste. Pela regra do próprio projeto, isso desqualifica os números anteriores de H1, H2 e H3, que precisam ser regerados com o pipeline corrigido. Um teste automatizado (`test_A1`) agora garante que trocar o conjunto de teste não altera a época escolhida nem o CVaR de validação.

11 Comparadores e desenho experimental

11.1 Dados e cenários

DREBIN-215: 15 036 aplicativos Android, 215 atributos binários de análise estática (permissões, *intents*, chamadas de API), $\approx 37\%$ malware. Divisão estratificada 80/20 para teste e, do restante, 80/20 para validação; a semente controla a divisão. Três cenários:

- **H1 — in-distribution:** divisão aleatória estratificada;
- **H2 — desbalanceamento:** o malware do *treino* é subamostrado para 5%, 2% e 1%; validação e teste mantêm a composição natural;
- **H3 — zero-day por cluster:** o malware é agrupado por *k*-means em 5 clusters no espaço de atributos; treina-se em 4 e testa-se no cluster retido (proxy conservador de “família nunca vista”, já que o DREBIN-215 não tem rótulos de família).

11.2 Os oito métodos

Todos recebem os mesmos dados, o mesmo espaço de busca ($\log_lr \in [-5, -2]$, $\log_wd \in [-6, -2]$, dropout $\in [0, 0.5]$, largura $\in [32, 256]$, profundidade $\in [1, 4]$) e, exceto ASHA, o mesmo orçamento de 28 treinamentos.

Método	Papel	O que isola
tune3_bestcvar	primário	ponto da fronteira com menor CVaR
tune3 (knee)	alternativo	a configuração “mais plana” razoável — se a geometria importa, deveria ganhar
bo_mono_cvar (B4)	E1	mesma BO, mesmo trial interno, <i>só</i> CVaR como objetivo: separa “geometria” de “BO explora bem”
tune3_placebo_bestcvar	E1	2º objetivo = ruído $\mathcal{N}(0, 1)$: se empata com o Tune3, o ganho é da exploração bi-objetivo
tune3_noddkf_bestcvar	E2	DDKF desligado (lr fixo): isola o nível MICRO
random_search	baseline	controle ingênuo
asha	baseline	<i>successive halving</i> (HPO moderno, orçamento em épocas)
sam	baseline	planura no espaço dos <i>pesos</i> (SAM, $\rho = 0.05$): o contraste direto com planura via hiperparâmetros

11.3 Protocolo estatístico (pré-registrado)

Comparações pareadas por semente (mesmos splits para todos os métodos); Wilcoxon signed-rank bilateral; correção de Holm sobre a família de comparações; tamanho de efeito d_z de Cohen (pareado) com intervalo de confiança bootstrap BCa (10 000 reamostras); **vitória pré-registrada** $\Leftrightarrow p_{\text{Holm}} < 0.05$ e $|d_z| \geq 0.30$ e direção favorável. Em H3, a unidade estatística é a semente (média sobre os 5 folds); a análise agrupando folds×sementes é reportada apenas como exploratória, porque sementes do mesmo fold compartilham o conjunto de teste.

Uma nota sobre poder: o Wilcoxon bilateral com n pares tem p mínimo $2/2^n$; com $n < 6$ é *impossível* atingir $p < 0,05$. Pilotos de 3 sementes validam o pipeline, não produzem significância. O protocolo usa 20 sementes (10 em H3), com ampliação única para 40 pré-declarada para casos inconclusivos.

12 Estado da evidência e a tensão científica

12.1 O que foi observado (com o pipeline anterior)

Hipótese	Cenário	Resultado
H1	in-distribution	sem vantagem significativa do Tune3
H2	5% / 2% / 1%	direcional a favor de best-CVaR, sem significância ($d_z = 0,04-0,20$)
H3	zero-day	best-CVaR $\succ$ RS, ASHA e SAM, $p_{\text{Holm}} < 0,001$; $ d_z = 0,46$ (RS/SAM) e $0,88$ (ASHA)

Validade

Esses números foram produzidos com a avaliação final que escolhia a época no conjunto de teste (Seção 10). Eles orientam expectativas, mas **não podem ser reportados**. A reexecução com o pipeline corrigido é o primeiro item do protocolo pré-registrado.

12.2 As três rachaduras na explicação geométrica

Mesmo tomando H3 pelo valor de face, a explicação “o Tune3 ganha *porque* encontra mínimos planos” tem três problemas:

1. **Correlação fraca, no cenário errado.** A correlação entre CVaR e curvatura medida foi $r = +0,26$ ($n = 60$), e foi medida em H1 — exatamente onde a teoria *não* prevê vantagem. Em H3 ela nunca foi medida.
2. **A anomalia do knee.** O ponto *mais plano* da fronteira (knee) tem CVaR consistentemente *pior* que o best-CVaR. Se planura causasse baixo risco de cauda, o knee deveria ganhar. Ele perde.
3. **O comparador decisivo nunca rodou.** Sem B4 (BO mono-objetivo só em CVaR), não se distingue “o mecanismo geométrico funciona” de “otimização bayesiana explora melhor o espaço que RS/ASHA”.

12.3 H_G contra H_M

H_G (geométrica)	baixa curvatura <i>causa</i> baixo CVaR e robustez a variantes não vistas
H_M (exploração)	o ganho vem da exploração multi-objetivo do espaço de hiper-parâmetros; a curvatura é um segundo objetivo qualquer

A evidência disponível é *mais consistente com H_M* . Isso não é uma derrota: é a pergunta que a Linha 1 existe para responder.

12.4 Linha 1: E0, E1, E2 e a árvore de decisão

- **E0** — correlação *parcial* entre CVaR e $\log \text{Tr}(H^2)$ dentro dos folds de H3, controlando $\log \eta$ e $\log \lambda_{wd}$ (`scripts/analyze_e0.py`). H_G prevê correlação parcial positiva com IC excluindo zero. Só é válida sobre resultados gerados pelo pipeline corrigido.

- **E1** — B4 e placebo, mesmo orçamento. H_G prevê que o Tune3 vence *ambos*; H_M prevê empate com o placebo e/ou derrota para B4.
- **E2** — DDKF ligado vs. desligado, com a telemetria de engajamento. Se o DDKF quase nunca sai do regime A, o nível MICRO é declarado inerte, seja qual for o CVaR.

Cenário	Condição	Consequência
C1	Tune3 vence B4 e placebo; E0 positiva	H_G sobrevive $\rightarrow$ H4 (robustez adversarial)
C2	placebo empata com Tune3	reposicionar a tese em torno da exploração multi-objetivo
C3	empate geral	artigo de metodologia de avaliação
C4	B4 vence Tune3	parada obrigatória e reunião de coordenação

Nenhum dos quatro é fracasso; cada um leva a um artigo diferente. O que seria fracasso é escrever qualquer deles antes de os dados decidirem.

13 Literatura: o que cada artigo sustenta — e o que não sustenta

A tabela abaixo reproduz a leitura *corrigida* do corpus do projeto (auditoria de 07/09/2026), após a descoberta de que uma versão anterior do material de apoio continha uma referência inexistente e quatro atribuições indevidas.

Referência	Sustenta	Não sustenta
Sagun, Bottou, Le-Cun (2017) [5]	espectro da Hessiana degenerado: bulk em zero + poucos outliers	que os outliers dominem o traço em <i>qualquer</i> arquitetura (verificar no MLP do DREBIN: razão $\lambda_{\max}^2 / \text{Tr}(H^2)$)
Yao et al. (2018) [6]	correlação <i>empírica</i> entre espectro de H_θ e robustez, mediada pelo <i>batch size</i>	relação matemática $\lambda_{\max} \leftrightarrow$ sensibilidade; o teorema do artigo é sobre H_x (entrada), não H_θ ; com B fixo, o desenho não transfere ao Tune3
Hochreiter, Schmidhuber (1997) [7]	mínimos planos têm menor complexidade descritiva (MDL)	aplicabilidade direta a redes ReLU modernas (ver Dinh et al. [8])
Foret et al. (2021) [9]	SAM: min-max sobre θ melhora generalização em visão	que atuar em h produza o mesmo efeito — isso é a hipótese do Tune3
Zhang et al. (2023, ICCV) [10]	planura ajuda em generalização de domínio	transferência automática para zero-day em malware
Cohen et al. (2021) [11]	<i>progressive sharpening</i> e EoS em GD full-batch	comportamento sob SGD com minibatches (o caso do Tune3)
Ly, Gong (2025) [12]	paisagem multifractal; GD encontra mínimos planos <i>automaticamente</i>	que otimizadores convencionais falhem por causa da multifractalidade
Tsipras et al. (2019) [13]	trade-off robustez $\times$ acurácia (útil para a fronteira de Pareto)	— (não está no corpus de PDFs; avaliado por conhecimento do artigo)
Hutchinson (1990) [2]; PyHessian [3]	estimador de traço; ferramentas de espectro	—

Referência inexistente

“Gholami et al. (2019), *Anatomy of Hessian*” não existe. O PDF homônimo que circulou no corpus é de Sonbolstan e Ali Gholami (geofísica, inversão sísmica) e não tem relação com redes neurais. As referências corretas para o cálculo espectral são Hutchinson (1990) e PyHessian [3].

Terminologia. Usar *geométrico* para propriedades quantificáveis via curvatura (Hessiana, autovalores, norma de Frobenius); reservar *topológico* para invariantes sob deformação contínua. “Blindagem topológica” deve ser lida como “blindagem geométrica” — e, enquanto H_G não for confirmada, como *hipótese* de blindagem.

14 Glossário de símbolos

$\theta \in \mathbb{R}^W$	pesos da rede
$h \in \mathcal{H} \subset \mathbb{R}^5$	hiperparâmetros: $\log_{10} \eta$, $\log_{10} \lambda_{wd}$, dropout, largura, profundidade
L, ℓ	perda média; perdas por amostra
CVaR_γ , $\gamma = 0,95$	média dos 5% piores casos
H_θ , $\text{Tr}(H^2) = \ H\ _F^2$	Hessiana da perda nos pesos; curvatura agregada
v_i, M	sondas de Rademacher; número de sondas (5 no treino)
GSNR	$\ \bar{g}\ ^2 / \text{Tr}(\Sigma)$, razão sinal-ruído do gradiente
$x_t = \log \eta_t$, ρ , σ_η	estado do DDKF; $\rho = 0,95$, $\sigma_\eta = 0,05$
z_t	observação do DDKF: $(L^{tr}, L^{val}, \log \text{GSNR})$
P_{xz}, P_{zz}, K, u_t	covariâncias empíricas, ganho, correção escalar
$R^2 = S_u / S_\infty$	razão de informação; regimes A/B/C em 0,10 e 0,50
$k_\gamma = \sqrt{\gamma/(1-\gamma)}$	constante de Cantelli ($\approx 4,36$)
d_z, p_{Holm}	tamanho de efeito pareado; p corrigido

A Mapa teoria $\leftrightarrow$ código

Conceito	Arquivo	Classe / função
CVaR (Eq. 1)	tune3/core/objectives.py	cvar, cvar_rockafellar
Hutchinson (Eq. 3)	tune3/curvature/hutchinson.py	HutchinsonEstimator.estimate
GSNR	tune3/curvature/gsnr.py	GSNREstimator.from_gradients
DDKF (Eq. 4, 5)	tune3/micro/ddkf_controller.py	DDKFController.update, information_ratio
Regimes A/B/C	tune3/micro/regime_gating.py	RegimeGate
Cantelli	tune3/safety/cantelli.py	CantelliGuard.should_abort
EoS	tune3/safety/eos_detector.py	EoSDetector.update
Trial (Alg. 1)	tune3/integration/trial.py	run_trial
MACRO (GP + EHVI)	tune3/macro/bo_loop.py	Tune3MacroLoop
Placebo (E1)	tune3/integration/runner.py	Tune3RunConfig.objective2
B4 (E1)	tune3/baselines/bo_mono.py	MonoObjectiveB0
RS, ASHA, SAM	tune3/baselines/hpo.py, sam.py	—
Avaliação final	tune3/baselines/plain_trial.py	plain_trial(..., test_data=)
Protocolos H1 / S1 / S2	tune3/experiments/protocol.py, tune3/experiments/security_protocol.py	run_protocol, run_security_protocol
Estatística	tune3/experiments/stats.py	compare_paired, bootstrap_ci
E0	scripts/analyze_e0.py	—
Validação teoria $\times$ código	scripts/validate_theory.py	V1–V9

Referências

- [1] R. T. Rockafellar, S. Uryasev. Optimization of conditional value-at-risk. *Journal of Risk* 2(3):21–41, 2000.
- [2] M. F. Hutchinson. A stochastic estimator of the trace of the influence matrix for Laplacian smoothing splines. *Communications in Statistics — Simulation and Computation* 19(2):433–450, 1990.
- [3] Z. Yao, A. Gholami, K. Keutzer, M. W. Mahoney. PyHessian: Neural networks through the lens of the Hessian. *IEEE Int. Conf. on Big Data*, 2020 (arXiv:1912.07145).
- [4] J. Liu et al. Understanding why neural networks generalize well through GSNR of parameters. *ICLR*, 2020.

- [5] L. Sagun, L. Bottou, Y. LeCun. Eigenvalues of the Hessian in deep learning: singularity and beyond. arXiv:1611.07476, 2017.
- [6] Z. Yao, A. Gholami, Q. Lei, K. Keutzer, M. W. Mahoney. Hessian-based analysis of large batch training and robustness to adversaries. *NeurIPS*, 2018.
- [7] S. Hochreiter, J. Schmidhuber. Flat minima. *Neural Computation* 9(1):1–42, 1997.
- [8] L. Dinh, R. Pascanu, S. Bengio, Y. Bengio. Sharp minima can generalize for deep nets. *ICML*, 2017.
- [9] P. Foret, A. Kleiner, H. Mobahi, B. Neyshabur. Sharpness-aware minimization for efficiently improving generalization. *ICLR*, 2021.
- [10] X. Zhang et al. Flatness-aware minimization for domain generalization. *ICCV*, 2023.
- [11] J. Cohen, S. Kaur, Y. Li, J. Z. Kolter, A. Talwalkar. Gradient descent on neural networks typically occurs at the edge of stability. *ICLR*, 2021.
- [12] Ly, Gong. Optimization on multifractal loss landscapes explains a diverse range of geometrical and dynamical properties of deep learning. *Nature Communications*, 2025.
- [13] D. Tsipras, S. Santurkar, L. Engstrom, A. Turner, A. Madry. Robustness may be at odds with accuracy. *ICLR*, 2019.
- [14] S. Daulton, M. Balandat, E. Bakshy. Parallel Bayesian optimization of multiple noisy objectives with expected hypervolume improvement. *NeurIPS*, 2021.
- [15] S. Ament, S. Daulton, D. Eriksson, M. Balandat, E. Bakshy. Unexpected improvements to expected improvement for Bayesian optimization. *NeurIPS*, 2023.
- [16] L. Li et al. A system for massively parallel hyperparameter tuning (ASHA). *MLSys*, 2020.
- [17] D. Arp, M. Spreitzenbarth, M. Hübner, H. Gascon, K. Rieck. DREBIN: Effective and explainable detection of Android malware in your pocket. *NDSS*, 2014.
- [18] de Oliveira, Diniz, Kreutz, Schmidt, Mansilha. Tune3: Otimização multiestágio de hiperparâmetros com redução de custo e maior estabilidade em modelos de detecção de malware Android. *Anais do 9º WRSeg*, 2025 (menção honrosa).